C++ Programming

Modern Core & Advanced Developers Handbook

C++11 / C++17 / C++20 STANDARD

Document Title: C++ Fundamentals, OOP Architecture & STL Mastery

Target Audience: Software Engineers, Computer Science Students, Technical Interviewees

Key Topics: Memory Management, OOP, Pointers & References, Templates, STL Containers, Smart Pointers, C++11/17 Features

Author/Publisher: Educational Technical Series

Table of Contents

1. Introduction to C++ & Compilation Pipeline — Page 2
2. Basic Syntax, I/O Streams & Namespaces — Page 3
3. Data Types, Operators & Control Flow — Page 4
4. Functions, Overloading & Default Arguments — Page 5
5. Pointers, References & Dynamic Memory Allocation — Page 6
6. Object-Oriented Programming: Classes & Objects — Page 7
7. OOP Deep Dive: Inheritance, Polymorphism & Virtuals — Page 8
8. Operator Overloading & Copy/Move Semantics — Page 9
9. Templates & Generic Programming — Page 10
10. Standard Template Library (STL) Containers & Algorithms — Page 11
11. Smart Pointers, Modern C++ Features & Exception Handling — Page 12

1. Introduction to C++ & Compilation Pipeline

C++ is a high-performance, general-purpose programming language created by **Bjarne Stroustrup** in 1979 at Bell Labs as an extension of the C programming language. It provides high-level abstractions like Object-Oriented Programming (OOP) and generic templates while maintaining low-level memory access and hardware control.

Key Paradigms Supported by C++

- **Procedural Programming:** Function-based structured code execution inheriting directly from C.
- **Object-Oriented Programming (OOP):** Encapsulation, Abstraction, Inheritance, and Polymorphism.
- **Generic Programming:** Compile-time abstraction via templates allowing type-independent logic.
- **Functional Programming:** Lambda expressions, higher-order functions, and immutability primitives introduced in modern C++.

C++ Compilation Pipeline

Converting a C++ source code file (`.cpp`) into a native binary binary executable involves four distinct steps executed by compilers like GCC, Clang, or MSVC:

Stage	Input File	Output File	Tool / Action
1. Preprocessing	`main.cpp`	`main.i`	Evaluates `#include`, `#define`, removes comments, and applies conditional macros.
2. Compilation	`main.i`	`main.s`	Parses code, checks syntax, performs optimizations, and translates C++ into Assembly code.
3. Assembly	`main.s`	`main.o` / `main.obj`	Assembles Assembly code into relocatable native machine code (Object File).
4. Linking	`main.o`	`app.exe` / `a.out`	Links object files and static standard libraries to produce the executable binary.

Building C++ Applications via GCC (g++)

Modern developers use compilation flags to enforce code standard checks and optimize execution speed:

```
# Compile using C++17 standard with strict warnings enabled
g++ -std=c++17 -Wall -Wextra -O2 main.cpp -o my_app

# Execute the compiled binary
./my_app
```

> **C++ Standard Versions Evolution**
>
> C++98 → C++03 → **C++11** (Modern Era: auto, lambdas, smart pointers) → **C++14** → **C++17** (structured binding, optional) → **C++20** (concepts, ranges, coroutines) → **C++23**.

2. Basic Syntax, I/O Streams & Namespaces

Unlike C which uses `printf` and `scanf`, C++ introduces type-safe stream abstraction using standard input/output streams provided by `<iostream>`.

Anatomy of a Basic C++ Program

```cpp
#include <iostream>
#include <string>

// Explicit namespace declaration
using namespace std;

int main() {
    // Console Output using insertion operator (<<)
    cout << "Welcome to C++ Programming!" << endl;

    int age;
    cout << "Enter your age: ";
    // Console Input using extraction operator (>>)
    cin >> age;

    cout << "You are " << age << " years old." << endl;
    return 0;
}
```

Understanding Namespaces

Namespaces prevent identifier collisions in large software projects by creating modular scope boundaries.

```cpp
#include <iostream>

namespace Engine {
    void init() {
        std::cout << "Engine Initialized" << std::endl;
    }
}

namespace UI {
    void init() {
        std::cout << "UI System Initialized" << std::endl;
    }
}

int main() {
    // Call using scope resolution operator (::)
    Engine::init();
    UI::init();
    return 0;
}
```

> **Avoid `using namespace std;` in Header Files**
>
> Injecting full namespaces into global scope inside header files (`.h` / `.hpp`) causes pollution and dangerous name collisions across files. Prefer explicit scoping like `std::cout` or function-level directives.

3. Data Types, Operators & Control Flow

C++ is a strongly typed static language. Every variable must have a declared type before execution.

Primitive Data Types & Memory Sizes

Data Type Group	Keyword	Size (Bits/ Bytes)	Value Range / Description
Boolean	`bool`	1 Byte	`true` (1) or `false` (0)
Character	`char`	1 Byte	ASCII characters ('A', 'z', '9')
Integer	`int`	4 Bytes	-2,147,483,648 to 2,147,483,647
Floating Point	`float`	4 Bytes	Single precision (~7 decimal digits)
Double Float	`double`	8 Bytes	Double precision (~15 decimal digits)
Type Inference	`auto` (C++11)	Deduced	Compiler automatically resolves variable type at compile-time.

Modern C++ Type Deduction (`auto`)

```
auto count = 10;          // Deduuced as int
auto pi = 3.14159;        // Deduced as double
auto name = "Developer";  // Deduced as const char*
```

Control Flow Constructs

```
// Conditional Branching
int score = 88;
if (score >= 90) {
    std::cout << "Grade: A" << std::endl;
} else if (score >= 80) {
    std::cout << "Grade: B" << std::endl;
} else {
    std::cout << "Grade: C" << std::endl;
}

// Range-Based For Loop (Modern C++11)
int numbers[] = {10, 20, 30, 40, 50};
for (auto val : numbers) {
    std::cout << "Value: " << val << std::endl;
}
```

4. Functions, Overloading & Default Arguments

Functions divide complex tasks into reusable logic units. C++ enhances procedural functions with parameter default values and compile-time function overloading.

Function Overloading

Multiple functions can share the exact same name provided their parameter signatures (types or quantity) differ.

```cpp
#include <iostream>

int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}

int main() {
    std::cout << add(5, 10) << std::endl;        // Calls int add(int, int)
    std::cout << add(2.5, 4.3) << std::endl;    // Calls double add(double, double)
    std::cout << add(1, 2, 3) << std::endl;    // Calls int add(int, int, int)
    return 0;
}
```

Default Argument Parameters

```cpp
// Function declaration with default arguments
void logMessage(std::string msg, std::string level = "INFO", bool toFile = false);

void logMessage(std::string msg, std::string level, bool toFile) {
    std::cout << "[" << level << "] " << msg << " (File: " << toFile << ")" << std::endl;
}

int main() {
    logMessage("System starting...");                // Uses default level & toFile
    logMessage("Memory low warning!", "WARNING");    // Uses default toFile
    return 0;
}
```

5. Pointers, References & Dynamic Memory Allocation

Memory management is one of C++'s core strengths. C++ offers direct pointer operations alongside safer lvalue references.

Pointers vs References Comparison

Feature	Pointer (`T*`)	Reference (`T&`)
Re-assignment	Can be pointed to a different variable later.	Bound permanently at initialization; cannot re-bind.
Null Values	Can be `nullptr`.	Cannot be null; must refer to valid storage.
Syntax	Requires explicit dereferencing (`*ptr`).	Transparent syntax (behaves like an alias).
Memory Address	Has its own separate memory address.	Shares the exact same memory address as referent.

Code Demonstration: Pass by Reference

```cpp
#include <iostream>

void incrementByValue(int x) { x++; }      // Original variable unchanged
void incrementByPointer(int* x) { (*x)++; } // Mutates original variable
void incrementByRef(int& x) { x++; }       // Cleanest C++ idiomatic approach

int main() {
    int val = 10;
    incrementByRef(val);
    std::cout << "Updated Value: " << val << std::endl; // 11
    return 0;
}
```

Dynamic Memory Allocation (`new` / `delete`)

In C++, dynamic heap allocation uses type-safe operators `new` and `delete` instead of C's `malloc` and `free`.

```cpp
// Single element allocation
int* pScalar = new int(42);
delete pScalar; // Deallocate memory
pScalar = nullptr;

// Dynamic array allocation
int size = 5;
int* pArray = new int[size];
delete[] pArray; // MUST use delete[] for array blocks
pArray = nullptr;
```

6. Object-Oriented Programming: Classes & Objects

Classes form the backbone of C++ Object-Oriented software architectures, encapsulating attributes (data members) and actions (member functions).

Access Specifiers

- `public` : Accessible from anywhere outside the class boundary.
- `private` : Accessible only by member functions and friend classes.
- `protected` : Accessible inside class members and derived subclasses.

Class Implementation Example

```cpp
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accountNumber;
    double balance;

public:
    // Parameterized Constructor
    BankAccount(std::string accNum, double initialBalance)
        : accountNumber(accNum), balance(initialBalance) {} // Member Initializer List

    // Deposit Method
    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    // Const Member Function (Guarantees no state mutation)
    double getBalance() const {
        return balance;
    }
};

int main() {
    BankAccount myAcc("ACC-9876", 1500.00);
    myAcc.deposit(500.00);
    std::cout << "Current Balance: $" << myAcc.getBalance() << std::endl;
    return 0;
}
```

7. OOP Deep Dive: Inheritance, Polymorphism & Virtuals

Inheritance allows classes to reuse logic from parent classes, while polymorphism enables runtime dynamic binding.

Virtual Functions & Dynamic Polymorphism

```cpp
#include <iostream>

// Base Abstract Class
class Shape {
public:
    // Pure Virtual Function (Creates Interface)
    virtual void draw() const = 0;

    // Virtual Destructor (Essential for safe polymorphic cleanup)
    virtual ~Shape() {}
};

class Circle : public Shape {
public:
    void draw() const override {
        std::cout << "Drawing Circle Vector..." << std::endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() const override {
        std::cout << "Drawing Rectangle Geometry..." << std::endl;
    }
};

int main() {
    Shape* s1 = new Circle();
    Shape* s2 = new Rectangle();

    s1->draw(); // Dynamic dispatch executes Circle::draw()
    s2->draw(); // Dynamic dispatch executes Rectangle::draw()

    delete s1;
    delete s2;
    return 0;
}
```

> **Always Declare Virtual Destructors in Base Classes**
>
> If a class contains any `virtual` functions, its destructor MUST be declared `virtual`. Failing to do so causes undefined behavior and memory leaks when deleting derived objects through base pointers.

8. Operator Overloading & Copy/Move Semantics

Operator overloading empowers user-defined types to respond to custom operations (e.g., `+`, `==`, `<<`).

Operator Overloading & Move Semantics

```cpp
#include <iostream>

class Vector2D {
public:
    double x, y;

    Vector2D(double x = 0, double y = 0) : x(x), y(y) {}

    // Overloading Binary '+' Operator
    Vector2D operator+ (const Vector2D& other) const {
        return Vector2D(x + other.x, y + other.y);
    }

    // Overloading Stream Output '<<' Operator
    friend std::ostream& operator<< (std::ostream& os, const Vector2D& vec) {
        os << "(" << vec.x << ", " << vec.y << ")";
        return os;
    }
};

int main() {
    Vector2D v1(3.5, 2.0), v2(1.5, 4.0);
    Vector2D v3 = v1 + v2;
    std::cout << "Vector Result: " << v3 << std::endl; // (5.0, 6.0)
    return 0;
}
```

Rule of Five (Modern C++ Resource Management)

If your class manually manages dynamic memory resources, explicitly define or delete the following 5 special member functions:

1. **Destructor:** `~Class()`
2. **Copy Constructor:** `Class(const Class& other)`
3. **Copy Assignment Operator:** `Class& operator=(const Class& other)`
4. **Move Constructor (C++11):** `Class(Class&& other) noexcept`
5. **Move Assignment Operator (C++11):** `Class& operator=(Class&& other) noexcept`

9. Templates & Generic Programming

Templates allow developers to write generic, type-independent logic compiled efficiently via static code generation.

Function and Class Templates

```cpp
#include <iostream>
#include <string>

// Function Template
template <typename T>
T findMax(T a, T b) {
    return (a > b) ? a : b;
}

// Generic Class Template
template <typename K, typename V>
class KeyValuePair {
private:
    K key;
    V value;
public:
    KeyValuePair(K k, V v) : key(k), value(v) {}
    void print() const {
        std::cout << key << " => " << value << std::endl;
    }
};

int main() {
    std::cout << findMax(10, 20) << std::endl;         // Deduces T = int
    std::cout << findMax(4.5, 2.1) << std::endl;       // Deduces T = double

    KeyValuePair<std::string, int> studentScore("John", 95);
    studentScore.print();
    return 0;
}
```

10. Standard Template Library (STL) Containers & Algorithms

The Standard Template Library (STL) provides robust, battle-tested algorithms, iterators, and data structures.

Core STL Container Taxonomy

Category	Container	Internal Data Structure	Time Complexity Key Ops
Sequence	`std::vector<T>`	Dynamic Contiguous Array	Random Access: $O(1)$, Append: $O(1)$ amortized
Sequence	`std::list<T>`	Doubly Linked List	Insert/Delete: $O(1)$, Access: $O(N)$
Associative	`std::map<K, V>`	Red-Black Balanced Tree	Search/Insert/Delete: $O(\log N)$
Unordered	`std::unordered_map<K, V>`	Hash Table	Average Search: $O(1)$, Worst: $O(N)$

STL Operations & Algorithms Example

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> vec = {40, 10, 50, 20, 30};

    // 1. Sorting using std::sort
    std::sort(vec.begin(), vec.end());

    // 2. Algorithms with Lambdas (C++11)
    int target = 30;
    auto it = std::find(vec.begin(), vec.end(), target);

    if (it != vec.end()) {
        std::cout << "Found " << *it << " in vector!" << std::endl;
    }

    return 0;
}
```

11. Smart Pointers, Modern C++ Features & Exception Handling

Modern C++ (C++11 onwards) eliminates manual dynamic allocation memory leaks using Smart Pointers (RAII principle).

Smart Pointers (`<memory>`)

- `std::unique_ptr<T>`: Exclusive ownership pointer. Cannot be copied, only moved.
- `std::shared_ptr<T>`: Shared reference-counted pointer. Memory freed when reference count hits 0.
- `std::weak_ptr<T>`: Non-owning observer reference to prevent circular `shared_ptr` memory leaks.

```cpp
#include <iostream>
#include <memory>

class Resource {
public:
    Resource() { std::cout << "Resource Acquired
"; }
    ~Resource() { std::cout << "Resource Auto-Destroyed
"; }
    void execute() { std::cout << "Executing Work...
"; }
};

int main() {
    // Modern dynamic ownership creation using std::make_unique
    std::unique_ptr<Resource> res = std::make_unique<Resource>();
    res->execute();
    return 0; // Automatically freed when going out of scope - Zero memory leaks!
}
```

Exception Handling Mechanics

```cpp
#include <iostream>
#include <stdexcept>

double divide(double a, double b) {
    if (b == 0.0) {
        throw std::runtime_error("Math Error: Division by Zero!");
    }
    return a / b;
}

int main() {
    try {
        std::cout << divide(10.0, 2.0) << std::endl;
        std::cout << divide(5.0, 0.0) << std::endl;
    } catch (const std::exception& e) {
        std::cerr << "Caught Exception: " << e.what() << std::endl;
    }
    return 0;
}
```

Best Practices for Modern C++ Development

- Prefer `std::unique_ptr` and `std::shared_ptr` over raw `new` / `delete` operators.
- Pass large objects by constant reference (`const T&`) to eliminate runtime copying overhead.
- Mark member functions that don't alter instance state as `const`.
- Use `override` keyword explicitly on derived virtual member functions for compile-time safety checks.